



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение

высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра математического обеспечения и стандартизации информационных технологий

Отчет по практической работе №2

по дисциплине «Тестирование и верификация программного обеспечения»

Выполнили:

Студенты группы ИКБО-32-23

Погосян Г.С.

Команда «ВМР»

Ильичев Г.П.

Проверил:

Отчет представлен

«__»_____2025г.

Москва 2025

Содержание

Цель работы.....	3
Задачи	3
Описание функциональности.....	3
Исходный код модуля (с преднамеренной ошибкой)	4
Модульные тесты (pytest).....	7
Исправление ошибки	9
Мутационное тестирование.....	14
Результаты.....	15
Вывод	17
Список используемой литературы.....	18

Цель работы

Познакомиться с методами модульного тестирования и мутационного тестирования на примере небольшого программного модуля — генератора паролей с возможностью проверки сложности и подсчёта энтропии.

Задачи

1. Разработать модуль на Python с минимум 5 функциями, одна из которых содержит преднамеренную ошибку (для роли тестировщика).
2. Написать модульные тесты (pytest) для проверки функциональности модуля и выявления ошибки.
3. Исправить ошибку, повторно запустить тесты.
4. Провести (описательно) мутационное тестирование с использованием MutPy (инструкции и пример команд).

Описание функциональности

Модуль реализует:

- Формирование алфавита символов на основе флагов (строчные, заглавные, цифры, специальные символы)
- Генерацию пароля с гарантированным включением минимум одного символа каждого выбранного типа
- Проверку соответствия требованиям (длина, наличие символов требуемых типов)
- Многофакторную оценку сложности на основе длины, разнообразия символов и отсутствия слабых паттернов
- Сравнение безопасности пароля с заданным минимальным уровнем через числовую шкалу силы

Исходный код модуля (с преднамеренной ошибкой)

```
import random
import string
import re
def generate_password(length=12, use_lower=True, use_upper=True, use_digits=True,
use_special=True):
    character_pool = ""
    if use_lower:
        character_pool += string.ascii_lowercase
    if use_upper:
        character_pool += string.ascii_uppercase
    if use_digits:
        character_pool += string.digits
    if use_special:
        character_pool += "!@#$%^&*()_+~[]{}|;:,<.>?"
    if not character_pool:
        raise ValueError("Хотя бы один набор символов должен быть включен")
    password = []
    if use_lower:
        password.append(random.choice(string.ascii_lowercase))
    if use_upper:
        password.append(random.choice(string.ascii_uppercase))
    if use_digits:
        password.append(random.choice(string.digits))
    if use_special:
        password.append(random.choice("!@#$%^&*()_+~[]{}|;:,<.>?"))
    remaining_length = length - len(password)
    for _ in range(remaining_length):
        password.append(random.choice(character_pool))
    random.shuffle(password)
    return "".join(password)
def check_password_strength(password):
    score = 0
    feedback = []

    if len(password) >= 12:
        score += 2
    elif len(password) >= 8:
        score += 1
    else:
        feedback.append("Пароль слишком короткий")
    has_lower = bool(re.search(r'[a-z]', password))
    has_upper = bool(re.search(r'[A-Z]', password))
    has_digit = bool(re.search(r'\d', password))
    has_special = bool(re.search(r'![@#$%^&*()_+~\[\]\{\}\|;:,<.>?]', password))
```

```

diversity_count = sum([has_lower, has_upper, has_digit, has_special])
if diversity_count == 4:
    score += 3 # Правильно
elif diversity_count == 3:
    score += 2 # Правильно
elif diversity_count == 2:
    score += 1 # Правильно
elif diversity_count == 1:
    score += 0
if has_lower and has_upper:
    score += 1
if score >= 6:
    strength = "очень сильный"
elif score >= 4:
    strength = "сильный"
elif score >= 2:
    strength = "средний"
else:
    strength = "слабый"
if re.search(r'(\1{2,}', password):
    score -= 1
    feedback.append("Обнаружены повторяющиеся символы")
if re.search(r'(abc|123|xyz)', password.lower()):
    score -= 1
    feedback.append("Обнаружены простые последовательности")

return {
    'score': max(0, score),
    'strength': strength,
    'length': len(password),
    'has_lower': has_lower,
    'has_upper': has_upper,
    'has_digit': has_digit,
    'has_special': has_special,
    'feedback': feedback
}

def validate_password_requirements(password, min_length=8, require_lower=True,
                                   require_upper=True, require_digits=True, require_special=True):
    if len(password) < min_length:
        return False, f"Пароль должен содержать не менее {min_length} символов"
    errors = []
    if require_lower and not re.search(r'[a-z]', password):
        errors.append("строчные буквы")
    if require_upper and not re.search(r'[A-Z]', password):
        errors.append("заглавные буквы")

```

```

if require_digits and not re.search(r'\d', password):
    errors.append("цифры")
if require_special and not re.search(r'[@#$%^&*()_+!-=\[\]\{\};:,.<>?]', password):
    errors.append("специальные символы")
if errors:
    return False, f"Отсутствуют: {'', '.join(errors)}"
return True, "Пароль соответствует требованиям"
def get_password_guidelines():
    return {
        "min_length": 8,
        "recommended_length": 12,
        "require_lowercase": True,
        "require_uppercase": True,
        "require_digits": True,
        "require_special": True,
        "description": "Используйте комбинацию букв разного регистра, цифр и
специальных символов"
    }
def generate_multiple_passwords(count=5, **kwargs):
    return [generate_password(**kwargs) for _ in range(count)]
def is_password_secure(password, min_strength="средний"):
    strength_map = {
        "слабый": 1,
        "средний": 2,
        "сильный": 3,
        "очень сильный": 4
    }
    result = check_password_strength(password)
    current_strength = result['strength']
    required_strength = min_strength
    return current_strength >= required_strength

```

Модульные тесты (pytest)

```
import pytest
import password_generator as pg
class TestPasswordGenerator:
    def test_generate_password_default(self):
        password = pg.generate_password()
        assert len(password) == 12
        assert any(c.islower() for c in password)
        assert any(c.isupper() for c in password)
        assert any(c.isdigit() for c in password)
        assert any(c in "!@#$%^&*()_+~[]{}|;:.,<>?" for c in password)
    def test_generate_password_custom_length(self):
        for length in [8, 10, 15]:
            password = pg.generate_password(length=length)
            assert len(password) == length
    def test_generate_password_character_sets(self):
        password = pg.generate_password(use_upper=False, use_digits=False,
use_special=False)
        assert all(c.islower() for c in password)
        password = pg.generate_password(use_lower=False, use_upper=False,
use_special=False, length=6)
        assert all(c.isdigit() for c in password)
        assert len(password) == 6
    def test_generate_password_no_character_sets(self):
        with pytest.raises(ValueError):
            pg.generate_password(use_lower=False, use_upper=False, use_digits=False,
use_special=False)
    def test_check_password_strength_strong(self):
        """Тест проверки сильного пароля."""
        strong_password = "SecurePass123!"
        result = pg.check_password_strength(strong_password)
        assert result['score'] >= 4
        assert result['strength'] in ["сильный", "очень сильный"]
        assert result['has_lower'] == True
        assert result['has_upper'] == True
        assert result['has_digit'] == True
        assert result['has_special'] == True
    def test_check_password_strength_weak(self):
        weak_password = "abc"
        result = pg.check_password_strength(weak_password)
        assert result['score'] <= 2
        assert result['strength'] == "слабый"
        assert len(result['feedback']) > 0
```

```

def test_check_password_strength_edge_cases(self):
    long_simple_password = "a" * 15
    result = pg.check_password_strength(long_simple_password)
    print(f"Long simple password score: {result['score']}") # Для отладки
    diverse_short = "Aa1!"
    result2 = pg.check_password_strength(diverse_short)
    print(f"Short diverse password score: {result2['score']}") # Для отладки
def test_validate_password_requirements(self):
    valid, message = pg.validate_password_requirements("SecurePass123!")
    assert valid == True
    assert "соответствует" in message
    valid, message = pg.validate_password_requirements("short")
    assert valid == False
    assert "не менее" in message
    valid, message = pg.validate_password_requirements("nouppercase123!")
    assert valid == False
    assert "заглавные" in message.lower()
def test_password_guidelines(self):
    guidelines = pg.get_password_guidelines()
    assert 'min_length' in guidelines
    assert 'recommended_length' in guidelines
    assert 'description' in guidelines
def test_generate_multiple_passwords(self):
    passwords = pg.generate_multiple_passwords(count=3, length=10)
    assert len(passwords) == 3
    for password in passwords:
        assert len(password) == 10
def test_is_password_secure(self):
    strong_result = pg.is_password_secure("SecurePass123!", "средний")
    assert strong_result == True
    weak_result = pg.is_password_secure("abc", "средний")
    assert weak_result == False
    weak_password = "password123" # Только строчные + цифры
    result = pg.is_password_secure(weak_password, "средний")
    print(f"Weak password secure result: {result}") # Для отладки
if __name__ == "__main__":
    test = TestPasswordGenerator()
    print("=== ТЕСТ ПРОВЕРКИ СЛОЖНОСТИ ПАРОЛЯ ===")
    test.test_check_password_strength_edge_cases()
    print("\n=== ТЕСТ БЕЗОПАСНОСТИ ПАРОЛЯ ===")
    test.test_is_password_secure()

```


Исправление ошибки

```
===== test session starts =====
platform win32 -- Python 3.13.3, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\artem\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\artem\Desktop\v
plugins: cov-7.0.0
collected 11 items

test_password_generator.py::TestPasswordGenerator::test_generate_password_default PASSED [ 9%]
test_password_generator.py::TestPasswordGenerator::test_generate_password_custom_length PASSED [ 18%]
test_password_generator.py::TestPasswordGenerator::test_generate_password_character_sets PASSED [ 27%]
test_password_generator.py::TestPasswordGenerator::test_generate_password_no_character_sets PASSED [ 36%]
test_password_generator.py::TestPasswordGenerator::test_check_password_strength_strong PASSED [ 45%]
test_password_generator.py::TestPasswordGenerator::test_check_password_strength_weak PASSED [ 54%]
test_password_generator.py::TestPasswordGenerator::test_check_password_strength_edge_cases PASSED [ 63%]
test_password_generator.py::TestPasswordGenerator::test_validate_password_requirements PASSED [ 72%]
test_password_generator.py::TestPasswordGenerator::test_password_guidelines PASSED [ 81%]
test_password_generator.py::TestPasswordGenerator::test_generate_multiple_passwords PASSED [ 90%]
test_password_generator.py::TestPasswordGenerator::test_is_password_secure FAILED [100%]

===== FAILURES =====
----- TestPasswordGenerator.test_is_password_secure -----
```

Отчет об ошибке #1:

Краткое описание: Неправильный подсчет очков сложности для паролей с одним типом символов

Статус: Open

Категория: Major

Тестовый случай: test_check_password_strength_edge_cases

Описание:

- Пароль "aaaaaaaaaaaaaaaa" (15 строчных букв) получает оценку сложности 2, что соответствует "среднему" уровню
- Ожидаемый результат: оценка 1 или 0 ("слабый" уровень)
- Проблема в строке 45-46 функции check_password_strength

Отчет об ошибке #2:

Краткое описание: Неправильное сравнение силы паролей в функции is_password_secure

Статус: Open

Категория: Critical

Тестовый случай: test_is_password_secure

Описание:

- Функция сравнивает строки "слабый", "средний" лексикографически вместо использования числовых значений
- "слабый" >= "средний" возвращает False из-за сравнения строк, что неправильно
- Ожидаемое поведение: сравнение на основе числовой шкалы силы

После замены все тесты должны проходить.

```
import random
import string
import re
import math
def generate_password(length=12, use_lower=True, use_upper=True, use_digits=True,
use_special=True):
    character_pool = ""
    if use_lower:
        character_pool += string.ascii_lowercase
    if use_upper:
        character_pool += string.ascii_uppercase
    if use_digits:
        character_pool += string.digits
    if use_special:
        character_pool += "!@#$%^&*()_+=[{}|;:,<.>?"
    if not character_pool:
        raise ValueError("Хотя бы один набор символов должен быть включен")
    password = []
    if use_lower:
        password.append(random.choice(string.ascii_lowercase))
    if use_upper:
        password.append(random.choice(string.ascii_uppercase))
    if use_digits:
        password.append(random.choice(string.digits))
    if use_special:
```

```

password.append(random.choice("!@#%&*()_+~[]{}|;:,<?"))
remaining_length = length - len(password)
for _ in range(remaining_length):
    password.append(random.choice(character_pool))

random.shuffle(password)
return "".join(password)

def check_password_strength(password):
    score = 0
    feedback = []
    if len(password) >= 12:
        score += 2
    elif len(password) >= 8:
        score += 1
    else:
        feedback.append("Пароль слишком короткий")
    has_lower = bool(re.search(r'[a-z]', password))
    has_upper = bool(re.search(r'[A-Z]', password))
    has_digit = bool(re.search(r'\d', password))
    has_special = bool(re.search(r'![@#%&*()_+~\[\]\{\}\|;:,<?]', password))
    diversity_count = sum([has_lower, has_upper, has_digit, has_special])
    # БЫЛО: Неправильная логика подсчета очков за разнообразие
    # if diversity_count == 4: score += 3
    # elif diversity_count == 3: score += 2
    # elif diversity_count == 2: score += 1
    # elif diversity_count == 1: score += 0 # ОШИБКА!
    # СТАЛО: Правильный подсчет очков за разнообразие
    score += diversity_count
    if has_lower and has_upper:
        score += 1
    # БЫЛО: Неправильные пороги для силы пароля
    # if score >= 6: strength = "очень сильный"
    # elif score >= 4: strength = "сильный"
    # elif score >= 2: strength = "средний" # ОШИБКА: должно быть >= 3
    # else: strength = "слабый"
    # СТАЛО: Правильные пороги для силы пароля
    if score >= 7:
        strength = "очень сильный"
    elif score >= 5:
        strength = "сильный"
    elif score >= 3:
        strength = "средний"
    else:
        strength = "слабый"
    if re.search(r'(\d|2|3|}', password):
        score -= 1
        feedback.append("Обнаружены повторяющиеся символы")
    if re.search(r'(abc|123|xyz)', password.lower()):

```

```

    score -= 1
    feedback.append("Обнаружены простые последовательности")
return {
    'score': max(0, score),
    'strength': strength,
    'length': len(password),
    'has_lower': has_lower,
    'has_upper': has_upper,
    'has_digit': has_digit,
    'has_special': has_special,
    'feedback': feedback
}
def validate_password_requirements(password, min_length=8, require_lower=True,
                                   require_upper=True, require_digits=True, require_special=True):
    if len(password) < min_length:
        return False, f"Пароль должен содержать не менее {min_length} символов"
    errors = []
    if require_lower and not re.search(r'[a-z]', password):
        errors.append("строчные буквы")
    if require_upper and not re.search(r'[A-Z]', password):
        errors.append("заглавные буквы")
    if require_digits and not re.search(r'\d', password):
        errors.append("цифры")
    if require_special and not re.search(r'[@#$$%^&*()_+!-=\[\]\{\}\|;,:.<>?]', password):
        errors.append("специальные символы")
    if errors:
        return False, f"Отсутствуют: {' '.join(errors)}"
    return True, "Пароль соответствует требованиям"
def get_password_guidelines():
    return {
        "min_length": 8,
        "recommended_length": 12,
        "require_lowercase": True,
        "require_uppercase": True,
        "require_digits": True,
        "require_special": True,
        "description": "Используйте комбинацию букв разного регистра, цифр и специальных
символов"
    }
def generate_multiple_passwords(count=5, **kwargs):
    return [generate_password(**kwargs) for _ in range(count)]
def is_password_secure(password, min_strength="средний"):
    strength_map = {
        "слабый": 1,
        "средний": 2,
        "сильный": 3,
        "очень сильный": 4
    }

```

```

result = check_password_strength(password)
current_strength = result['strength']
# БЫЛО: Неправильное сравнение строк лексикографически
# return current_strength >= required_strength # ОШИБКА!
# СТАЛО: Правильное сравнение числовых значений
current_level = strength_map.get(current_strength, 0)
required_level = strength_map.get(min_strength, 0)
return current_level >= required_level
def calculate_entropy(password):
    alphabet_size = 0
    if any(c.islower() for c in password):
        alphabet_size += 26
    if any(c.isupper() for c in password):
        alphabet_size += 26
    if any(c.isdigit() for c in password):
        alphabet_size += 10
    if any(c in "!@#%&*()_+~[]{}|;:,<>?" for c in password):
        alphabet_size += 20
    if alphabet_size == 0:
        return 0
    entropy = len(password) * math.log2(alphabet_size)
    return round(entropy, 2)

```

Мутационное тестирование

```
[mutant_001] rule=='==' to '!=' changes=5 -> KILLED
[mutant_002] rule=='>=' to '>' changes=11 -> KILLED
[mutant_003] rule=='>' to '>=' changes=16 -> KILLED
[mutant_004] rule=='<' to '<=' changes=6 -> KILLED
[mutant_005] rule='True to False' changes=13 -> KILLED
[mutant_006] rule='False to True' changes=2 -> KILLED
[mutant_007] rule='and to or' changes=5 -> KILLED
[mutant_008] rule='remove has_upper check' changes=1 -> KILLED
[mutant_009] rule='diversity count +1 -> -1' changes=1 -> KILLED
[mutant_010] rule='len>=12 -> len>=100' changes=1 -> KILLED
[mutant_011] rule='remove special chars from pool' changes=3 -> KILLED
[mutant_012] rule='regex simple sequence removal' changes=1 -> KILLED
```

=== SUMMARY ===

Mutants generated: 12

Mutants killed : 12

Mutants survived: 0

mutant_001	== to !=	KILLED	rc=1
mutant_002	>= to >	KILLED	rc=1
mutant_003	> to >=	KILLED	rc=1
mutant_004	< to <=	KILLED	rc=1
mutant_005	True to False	KILLED	rc=1
mutant_006	False to True	KILLED	rc=1
mutant_007	and to or	KILLED	rc=1
mutant_008	remove has_upper check	KILLED	rc=1
mutant_009	diversity count +1 -> -1	KILLED	rc=1
mutant_010	len>=12 -> len>=100	KILLED	rc=1
mutant_011	remove special chars from pool	KILLED	rc=1
mutant_012	regex simple sequence removal	KILLED	rc=1

PS C:\Users\artem\Desktop\p> |

Результаты

1. Разработан программный модуль генератора паролей.

Был создан модуль на языке Python, реализующий основные функции генерации и проверки паролей.

Программа умеет формировать пароль из различных наборов символов (строчные, заглавные, цифры, спецсимволы), проверять его сложность, вычислять уровень безопасности и энтропию.

2. В модуль преднамеренно внесена ошибка.

Ошибка касалась неверного подсчёта очков сложности пароля при малом разнообразии символов, а также некорректного сравнения строк при определении уровня безопасности.

Эти дефекты были выявлены с помощью модульных тестов.

3. Разработан комплекс модульных тестов (pytest).

Для всех функций были написаны тесты, проверяющие корректность работы при различных входных данных:

4. генерация паролей с разными наборами символов и длиной;
5. проверка минимальных требований к паролю;
6. оценка силы пароля в типичных и граничных случаях;
7. тестирование функции сравнения безопасности.

После запуска pytest ошибки были обнаружены и задокументированы.

8. Выполнено исправление ошибок.

После анализа тестов код был откорректирован:

9. исправлен алгоритм подсчёта баллов за разнообразие символов;
10. изменены пороги классификации сложности;
11. реализовано корректное числовое сравнение уровней безопасности.

После исправления все тесты успешно прошли.

12. Проведено мутационное тестирование.

Был создан собственный скрипт `manual_mutation_runner.py`, реализующий логику мутационного анализа.

В ходе эксперимента автоматически создавались изменённые версии исходного кода («мутанты»), после чего для каждой из них выполнялся запуск тестов.

Итогом стало определение, какие тесты смогли выявить изменения в коде.

Большинство мутантов были «убиты», что говорит о высоком качестве набора тестов.

Вывод

В результате выполнения работы получен корректно функционирующий модуль генератора паролей с системой автоматических тестов и возможностью оценки энтропии.

Все тесты после исправления ошибок выполняются успешно, что подтверждает высокую надёжность и качество кода.

Проведённое мутационное тестирование показало, что тестовый набор обладает высокой эффективностью и способен выявлять большинство возможных дефектов.

Список используемой литературы

- **Мартин Р.** Чистый код: создание, анализ и рефакторинг. — СПб.: Питер, 2021.
- **Соммервилл И.** Инженерия программного обеспечения. — 10-е изд. — М.: Вильямс, 2020.
- **Бойс Д., Браун Э.** Автоматизация тестирования программного обеспечения. — М.: ДМК Пресс, 2019
- **Документация Python (официальный сайт).** — Режим доступа: <https://docs.python.org/3/>
- **Документация Pytest (официальный сайт).** — Режим доступа: <https://docs.pytest.org/en/stable/>
- **Троелсен Э., Дзепикс Ф.** Язык программирования Python. Основы профессионального программирования. — М.: Вильямс, 2022.
- **РТУ МИРЭА.** Методические указания по выполнению практических работ по дисциплине «Тестирование и верификация программного обеспечения». — Москва: МИРЭА, 2024.